

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <conio.h>
#include "stack.h"

/* Define Tree Node */
typedef struct node {
    char data;
    struct node *left, *right;
} node;

/* Global Tree Stack for Turbo C */
node *tstack[50];
int ttop = -1;

void pushTree(node *n) {
    tstack[++ttop] = n;
}

node *popTree() {
    return tstack[ttop--];
}

/* Helper to create a new tree node */
node *newNode(char val) {
    node *temp = (node *)malloc(sizeof(node));
    temp->data = val;
    temp->left = NULL;
    temp->right = NULL;
    return temp;
}

/* Tree Printer with visual vertices */
void printTree(node *root, char *prefix, int isLeft) {
    char newPrefix[100];
    if (root == NULL) return;

    printf("%s", prefix);
    printf("%s", (isLeft ? "|-- " : "L-- "));
    printf("%c\n", root->data);

    /* Construct the next prefix string */
    strcpy(newPrefix, prefix);
    strcat(newPrefix, (isLeft ? "|  " : "  "));

    if (root->left && root->right) {
        printTree(root->left, newPrefix, 1);
        printTree(root->right, newPrefix, 0);
    }
}

```

```

        else if (root->left) {
            printTree(root->left, newPrefix, 0);
        }
    }

/* Specialized print for the parser stack */
void printParseStack(stack *s) {
    int i;
    for (i = 0; i <= s->top; i++) {
        printf("%c", (char)s->item[i]);
    }
}

/* Reduction Logic */
int reduce(stack *s, char *remaining) {
    /* F -> a */
    if (s->top >= 0 && s->item[s->top] == 'a') {
        node *F;
        pop(s);
        push(s, 'F');
        F = newNode('F');
        F->left = newNode('a');
        pushTree(F);
        printf("\n"); printParseStack(s); printf("\t%s\tReduce F->a", remaining);
        return 1;
    }

    /* T -> T * F */
    if (s->top >= 2 && s->item[s->top] == 'F' && s->item[s->top - 1] == '*' &&
s->item[s->top - 2] == 'T') {
        node *fNode, *tOld, *tNew, *op;
        pop(s); pop(s); pop(s);
        push(s, 'T');
        fNode = popTree();
        tOld = popTree();
        tNew = newNode('T');
        op = newNode('*');
        tNew->left = op;
        op->left = tOld;
        op->right = fNode;
        pushTree(tNew);
        printf("\n"); printParseStack(s); printf("\t%s\tReduce T->T*F", remaining);
        return 1;
    }

    /* T -> F */
    if (s->top >= 0 && s->item[s->top] == 'F') {
        node *fNode, *tNode;
        pop(s);
        push(s, 'T');
    }
}

```

```

        fNode = popTree();
        tNode = newNode('T');
        tNode->left = fNode;
        pushTree(tNode);
        printf("\n"); printParseStack(s); printf("\t%s\tReduce T->F", remaining);
        return 1;
    }

    /* E -> E + T */
    if (s->top >= 2 && s->item[s->top] == 'T' && s->item[s->top - 1] == '+' &&
s->item[s->top - 2] == 'E') {
        node *tNode, *eOld, *eNew, *op;
        pop(s); pop(s); pop(s);
        push(s, 'E');
        tNode = popTree();
        eOld = popTree();
        eNew = newNode('E');
        op = newNode('+');
        eNew->left = op;
        op->left = eOld;
        op->right = tNode;
        pushTree(eNew);
        printf("\n"); printParseStack(s); printf("\t%s\tReduce E->E+T", remaining);
        return 1;
    }

    /* E -> T (lookahead check) */
    if (s->top >= 0 && s->item[s->top] == 'T' && remaining[0] != '*') {
        node *tNode, *eNode;
        pop(s);
        push(s, 'E');
        tNode = popTree();
        eNode = newNode('E');
        eNode->left = tNode;
        pushTree(eNode);
        printf("\n"); printParseStack(s); printf("\t%s\tReduce E->T", remaining);
        return 1;
    }
    return 0;
}

void main() {
    stack s;
    char input[50];
    int i = 0;

    clrscr(); /* Turbo C++ specific */
    createstack(&s);

    printf("Grammar:\nE -> E+T | T\nT -> T*F | F\nF -> a\n\n");

```

```

printf("Enter string: ");
scanf("%s", input);

printf("\nStack\tInput\tAction\n");
push(&s, '$');

while (1) {
    if (i < (int)strlen(input)) {
        push(&s, input[i]);
        printf("\n"); printParseStack(&s);
        printf("\t%s\tShift", input + i + 1);
        i++;
    }

    while (reduce(&s, input + i));

    if (i >= (int)strlen(input) && s.top == 1 && s.item[0] == '$' && s.item[1]
== 'E') {
        printf("\n$E\t$\tAccept\n");
        printf("\nParse Tree:\n");
        printTree(tstack[ttop], "", 0);
        break;
    }

    if (i >= (int)strlen(input)) {
        printf("\nReject\n");
        break;
    }
}
getch(); /* Pause to see result in Turbo C */
}

```

```

/*
Grammar:
E -> E+T | T
T -> T*F | F
F -> a

```

Enter string: a+a+a

```

$E+T    +a      Reduce T->F
$E       +a      Reduce E->E+T
$E+     a        Shift
$E+a    Shift

```

\$E+F		Reduce	F $\rightarrow$ a
\$E+T		Reduce	T $\rightarrow$ F
\$E		Reduce	E $\rightarrow$ E+T
\$E	\$	Accept	

Parse Tree:

```
L-- E
L-- +
|   |
|   |--- E
|   L-- +
|   |   |
|   |   |--- E
|   |   L-- T
|   |       |
|   |       |--- F
|   |           |
|   |           |--- a
|   |           L-- a
|   |       L-- F
|   |       L-- a
|   L-- T
|   L-- F
|   L-- a
```

 $\ast/$
$$/^{*}$$

Grammar:

$$E \rightarrow E+T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow a$$

Enter string: a+a*a

Stack	Input	Action
\$a	+a*a	Shift
\$F	+a*a	Reduce F->a
\$T	+a*a	Reduce T->F
\$E	+a*a	Reduce E->T
\$E+	a*a	Shift
\$E+a	*a	Shift
\$E+F	*a	Reduce F->a
\$E+T	*a	Reduce T->F
\$E	*a	Reduce E->E+T
\$E*	a	Shift
\$E*a		Shift
\$E*F		Reduce F->a
\$E*T		Reduce T->F
\$E*E		Reduce E->T
Reject		

 $\ast/$